Project Overview:

The project is a full-stack **Airbnb-like** web application, allowing users to browse, create, and manage property listings. The app also includes features such as user authentication, property image uploads, and interactive maps displaying property locations. Here's a breakdown of the components:

Technologies Used:

- **Node.js**: Server-side runtime environment to handle backend operations.
- **Express.js**: Framework for building the application using the MVC pattern.
- **MongoDB Atlas**: Database service to store user information, property details, and bookings.
- **EJS**: Templating engine for dynamic HTML rendering.
- **Mongoose**: ODM to interact with MongoDB.
- **Cloudinary**: For storing and managing images uploaded by users.
- **Mapbox**: For displaying interactive maps and geocoding addresses to show property locations.
- **Passport.js**: For user authentication (signup, login, and logout).
- **Multer**: Middleware to handle file uploads.
- **Joi**: For server-side input validation.
- **Session & Flash**: For managing user sessions and displaying real-time feedback messages.

Key Features:

1. **User Authentication (Passport.js)**:
    - Users can sign up, log in, and log out. I implemented user sessions with **express-session** to maintain logged-in status across pages.
    - Used Passport.js to authenticate users securely, supporting local strategies for login.
2. **Property Listings (CRUD Operations)**:
    - **Create**: Logged-in users can create new property listings, providing details such as title, description, price, location, and images.
    - **Read**: All users can browse property listings, view property details, and see each location on an interactive map (Mapbox).

- **Update**: Users can edit their property details.
- **Delete**: Users can delete their property listings.

3. **Image Upload (Cloudinary + Multer)**:
    - **Multer** was used to handle form data and file uploads.
    - The images were uploaded to **Cloudinary** for cloud storage. Cloudinary returns a URL that's stored in the MongoDB database and linked with each property listing.
4. **Interactive Maps (Mapbox)**:
    - I used **Mapbox** to show the property's location on an interactive map.
    - The location is geocoded using Mapbox's API, allowing users to see where the property is situated directly on the site.
5. **Form Validation (Joi)**:
    - Form data, such as user sign-up information and property details, were validated using **Joi**.
    - This ensured that all fields were correctly formatted, with appropriate error messages being shown when the input was invalid (handled by **Flash** messages).
6. **Real-Time Error & Success Handling (Flash + Sessions)**:
    - Implemented **Flash** messages to show users real-time feedback, like success messages when a listing is created or errors for invalid login attempts.

Challenges Faced & Solutions:

1. **Image Handling**:
    - **Challenge**: Uploading multiple property images while ensuring secure storage and efficient access.
    - **Solution**: Using **Multer** to handle file uploads from the forms, and integrating **Cloudinary** to store images securely. This made image handling smooth and scalable, as Cloudinary optimizes images for different screen sizes automatically.
2. **Dynamic Maps**:
    - **Challenge**: Displaying property locations on a map dynamically based on user input.

- **Solution**: Used **Mapbox** to generate an interactive map. By geocoding user-inputted addresses, I displayed precise property locations on the map, giving users a visual sense of where each listing is situated.

3. **Authorization for Listings**:
    - **Challenge**: Ensuring only the user who created a listing can edit or delete it.
    - **Solution**: Used Passport.js and session data to implement role-based access control. Validated the user's ownership of the listing before allowing any updates or deletions.
4. **Form Validation**:
    - **Challenge**: Properly validating user input, especially in forms with multiple fields.
    - **Solution**: Implemented **Joi** to enforce data validation rules on server-side requests, ensuring robust input handling to avoid incorrect or malicious data.
5. **Error & Success Feedback**:
    - **Challenge**: Communicating success and error messages back to users in real-time.
    - **Solution**: I used **Flash** messages in combination with **sessions** to provide immediate feedback to users when they performed actions like creating, updating, or deleting listings.
6. **Scalability**:
    - **Challenge**: Designing the application to handle more users and listings as it grows.
    - **Solution**: Using **MongoDB Atlas**, I ensured the database could scale horizontally with demand. By using **Cloudinary** to store images externally, I reduced server load and ensured better app performance.

Third-Party Services:

- **MongoDB Atlas**: Managed NoSQL database service for storing all user and property data. Atlas provided reliable, secure, and scalable storage for the project.
- **Cloudinary**: Cloud-based service to store and serve images. This eliminated the need for managing file storage on the server, optimizing performance and image handling.
- **Mapbox**: Used to display maps and geocode addresses, allowing users to visualize property locations. The Mapbox API made it simple to integrate map functionality.

- **Passport.js**: Authentication library for securing the app. It allowed users to securely log in, log out, and protect routes like property creation and deletion.
- **Multer**: For handling multipart form data and enabling users to upload images.
- **Joi**: Schema validation library used to ensure form data, such as property details and user information, met the required standards.

Conclusion:

This **Airbnb clone** project challenged me to work with a variety of third-party services, handle complex CRUD operations, and manage user authentication securely. By integrating tools like **Cloudinary** for image uploads, **Mapbox** for location services, and **Passport.js** for authentication, I built a robust, scalable web application using the **Node.js** and **Express.js** ecosystem. The project helped solidify my understanding of full-stack development, backend operations, and working with third-party services in a real-world application.

Project link :- https://wanderlust-mmuj.onrender.com (takes 1 min to open)